AI-Powered Road Hazard Detection

Full Development Brief and Technical Documentation

Version 1.0 | Free-only stack | Web-based implementation

Project Goal	Detect potholes, stray animals, and accidents from CCTV video, generate structured incident reports, calculate severity and priority, and notify authorities through a dashboard and email alerts.

1. Project Overview

Problem statement: Use existing CCTV feeds to detect road hazards in real time and report them to police / RTO / local authorities.

- Potholes: detect damaged road surfaces, classify severity, track before and after state, and geo-tag the report.
- Stray animals: detect dogs, cows, goats, pigs, and similar road hazards; classify behavior and risk.
- Accidents: detect abnormal motion, collisions, sudden stops, hit-and-run behavior, and traffic disruption.
- Authority reporting: create a structured incident record, send an email alert, and show the event in a live dashboard.

Primary output: A working web-based product with AI detection, incident storage, live dashboard, and authority reporting.

2. Non-Negotiable Constraints

- Use only free or open-source tools.
- Do not depend on paid APIs, credits, or billing-required services.
- Keep the stack simple enough for hackathon implementation.
- Prefer stable and widely used libraries over experimental or overly complex options.
- Keep the architecture deterministic so a fresh AI assistant does not hallucinate extra tools or unsupported steps.

3. Final Recommended Tech Stack

Layer	Choice	Purpose	Notes
Frontend	Next.js 14+ with TypeScript	Public dashboard and authority dashboard	App Router only
UI Styling	Tailwind CSS + shadcn/ui	Fast, clean, consistent interface	Avoid heavy UI kits
Charts / Maps	Recharts + Leaflet + OpenStreetMap	Analytics and geo-visualization	No paid map dependency
Backend API	Python 3.11 + FastAPI	AI service and event ingestion	Keep API responses simple
Computer Vision	OpenCV + Ultralytics YOLOv8	Frame processing and object detection	Default model: yolov8n
Tracking	Simple centroid tracking / short-window matching	Before-after matching and motion analysis	Avoid complex tracker setup
Database	Neon PostgreSQL + Prisma	Store incidents, cameras, status, and history	Use one clean relational schema
Email Alerts	Nodemailer + SMTP	Send authority alerts	Use free SMTP only
Storage	Local file storage during development	Save snapshots and clips	Keep it simple for demo
Realtime Updates	Polling or Server-Sent Events	Refresh dashboards automatically	Avoid WebSocket complexity if not needed

Recommended model rule: Use YOLOv8n as the default detection model for real-time speed. If accuracy is insufficient, move to YOLOv8s; do not start with a heavier model unless the hardware is already known.

4. System Architecture

High-level flow: CCTV / video source -> Python AI service -> incident JSON -> Next.js API / database -> email alert -> live dashboard.

Component	Responsibility	Output
CCTV / Video Input	Feeds live or recorded video into the system.	Frames to analyze
AI Service (FastAPI)	Runs detection, tracking, severity calculation, and event assembly.	Structured incident payload
Next.js Web App	Provides authority dashboard, admin views, and API routes.	Stored and displayed incidents
Neon PostgreSQL	Stores incidents, camera registry, snapshots, and status history.	Persistent records
Email Engine	Sends immediate alert emails to authorities.	Actionable notification
Dashboard	Shows live incidents, filters, analytics, and resolution state.	Decision support

5. Unified Incident Workflow

Standard runtime sequence: Video stream starts, frames are processed continuously, hazards are detected, incident state is updated, and the dashboard plus email channel are refreshed.

Step	Stage	Technical action
1	Video ingestion	CCTV stream, RTSP feed, or uploaded video is received by the AI service.
2	Frame extraction	OpenCV reads frames at a controlled rate such as 1 to 5 FPS for stable processing.
3	Detection	YOLOv8 detects potholes, animals, vehicles, and humans depending on the module.
4	Event logic	Hazard-specific rules convert raw detections into pothole, animal, or accident events.
5	Severity / risk	The system calculates severity, risk score, or priority label.
6	Record matching	The new event is matched against existing records using camera ID and spatial proximity.
7	Status update	The incident is created, updated, or marked resolved depending on repeated observations.
8	Geo-tagging	Camera location metadata is attached to the event.
9	Report generation	A structured report is stored in the database and sent by email.
10	Dashboard update	The authority dashboard refreshes and displays the event in real time or near real time.

6. Hazard-Specific Technical Workflows

6.1 Pothole Detection and Lifecycle Tracking

- Detect pothole bounding boxes from the video feed.
- Compute pothole severity using bounding-box area, road position, and repeated detection frequency.

- Store the first confirmed detection as the before state.
- On later detections, match the new detection with the existing pothole using camera ID and spatial distance.
- Update the after image and compare area changes to determine whether the pothole is growing, unchanged, or reduced.
- If the pothole is not detected for a configured duration, mark the incident as resolved, but keep manual override available in the authority dashboard.

Severity	Meaning	Operational response
Low	Minor road damage with limited immediate risk	Log and monitor
Medium	Visible road damage that can affect vehicles	Report and schedule repair
High	Dangerous pothole that can trigger accidents	Immediate authority alert

Pothole report fields: incident_id, camera_id, latitude, longitude, severity_label, severity_score, confidence, before_image, after_image, status, last_seen_at.

6.2 Stray Animal Detection and Behavior Risk

- Detect the relevant animal types for the Indian road context: dog, cow, goat, pig, and similar stray hazards.
- Classify behavior into standing, sitting, crossing, or running.
- Track motion across frames to determine whether the animal is approaching the road zone.
- Assign a risk score based on behavior and position within the frame.
- Use the road-zone rule: if an animal enters the road zone or its trajectory intersects the road boundary, create an alert.
- Record the hotspot location so repeated animal appearances at the same camera can be analyzed later.

Risk level	Behavior	Action
Low	Standing at roadside or not moving toward the road	Log only or low-priority warning
Medium	Slow movement or partial road entry	Warning on dashboard
High	Crossing, running, or herd movement on road	Immediate alert to authority dashboard and email

6.3 Accident Detection and Severity Analysis

- Use multi-frame motion analysis across a short window such as the previous 3 to 5 frames plus the current frame.
- Track vehicles and detect abnormal patterns such as overlap, collision, and sudden stop.
- Estimate severity using speed, number of vehicles, vehicle type, and human presence near the incident.
- Flag possible hit-and-run behavior when an involved vehicle leaves the scene immediately after the collision.
- Estimate traffic disruption using vehicle density before and after the accident.

Score	Severity	Meaning
0-30	Low	Minor event or low disruption

Score	Severity	Meaning
31-70	Medium	Moderate collision or traffic disruption
71-100	High	Serious accident requiring urgent response

Accident report fields: event_id, camera_id, involved_vehicle_count, severity_score, severity_label, hit_and_run_flag, congestion_level, snapshot_image, timestamp, status.

7. Data Model

Use a small relational schema in Neon PostgreSQL. Keep the tables simple and stable so the Next.js application and the AI service can both read the same incident structure.

Table	Core fields	Purpose	Notes
cameras	id, name, location_name, latitude, longitude, rtsp_url, active	Maps camera to a real place	Use manual mapping if GPS is unavailable
incidents	id, camera_id, hazard_type, severity_label, severity_score, status	Stores every hazard event	Single unified incident table
incident_snapshots	id, incident_id, image_url, frame_time, frame_type	Keeps before / after evidence	Optional multiple snapshots per incident
incident_history	id, incident_id, event_type, event_time, notes	Tracks lifecycle changes	Useful for audit trail
users	id, name, email, role, password_hash	Authority login	Only if authentication is required

8. API Contract

The AI service sends JSON events to the web application. Keep the payload schema stable so the fresh development chatbot does not invent new fields.

Method	Route	Purpose	Main payload fields
POST	/api/incidents	Create a new incident	type, severity, camera_id, confidence, latitude, longitude, images, metadata
GET	/api/incidents	List incidents	status, type, date, priority filters
GET	/api/incidents/:id	Read one incident	incident id
PUT	/api/incidents/:id	Update incident status	status, notes, resolved_at
GET	/api/cameras	List camera registry	camera mapping data
POST	/api/reports/daily	Generate summary report	date range, type filter

Incident payload shape:
{"type":"pothole","severity_label":"HIGH","severity_score":88,"camera_id":"CAM_01","latitude":19.87,"longitude":75.34,"confidence":0.91,"before_image":"...","after_image":"...","status":"reported"}

9. Dashboard Requirements

Authority dashboard: Live incident feed, filters by hazard type and severity, incident detail page, before/after images, map view, charts, acknowledge / resolve actions, and email history.

Public or demo dashboard: Optional read-only view showing current hazards on a map or table for presentation.

Analytics: Counts by hazard type, severity distribution, hotspot map, and recent activity timeline.

- Use Leaflet with OpenStreetMap for map display.
- Use Recharts for charts and summary cards.
- Use polling every few seconds or Server-Sent Events for live updates.
- Use a clean table layout on desktop and stacked cards on mobile.

10. Email and Authority Reporting

- Send an email immediately when a high-priority hazard is detected.
- Attach the snapshot image or include a secure image link.
- Include camera location, timestamp, hazard type, severity, and confidence score.
- Use a single consistent email template for potholes, animals, and accidents.
- Send a daily or weekly summary report only as an optional extension.

Field	Example
Subject	Road Hazard Alert - High Severity Pothole
Location	MG Road, Aurangabad
Timestamp	2026-03-26 15:30
Severity	High
Confidence	0.91
Action link	Map link or dashboard incident link

11. Recommended Build Order

1. Create the Next.js web app, set up Tailwind CSS, shadcn/ui, Prisma, and the dashboard layout.
2. Create the Neon PostgreSQL database and run the initial schema for cameras, incidents, snapshots, and history.
3. Build the authority dashboard pages and CRUD routes first so you can see live data immediately.
4. Build the Python FastAPI service and connect it to the video source.
5. Integrate YOLOv8 detection for potholes, animals, vehicles, and people.
6. Add the pothole severity engine, animal risk engine, and accident analysis engine.
7. Add the matching logic for before / after state and resolution detection.
8. Connect the AI service to the Next.js API route that stores incidents in Neon.
9. Add Nodemailer SMTP email alerts and dashboard notifications.
10. Finish with analytics, filters, maps, and presentation polish.

12. Clean Project Folder Structure

Suggested structure: Keep the AI service and the web application in separate folders so the responsibilities stay clear.

```
road-hazard-system/
  ai-service/
    app.py
    detector.py
```

```
    pothole_engine.py
    animal_engine.py
    accident_engine.py
    tracking.py
    config.py
    requirements.txt
  web-app/
    app/
      page.tsx
      dashboard/page.tsx
      incidents/page.tsx
      api/
        incidents/route.ts
        incidents/[id]/route.ts
        reports/daily/route.ts
    components/
    lib/
      db.ts
      mail.ts
      cameras.ts
    prisma/
      schema.prisma
    public/
      uploads/
    package.json
  docs/
    implementation_brief.docx
```

13. Fresh AI Chatbot Start Prompt

Use the following instruction when starting a new AI coding chat:

Build a web-based AI road hazard detection system using Next.js 14+ with TypeScript, Tailwind CSS, shadcn/ui, Prisma, and Neon PostgreSQL for the web layer, and Python 3.11 with FastAPI, OpenCV, and Ultralytics YOLOv8n for the AI layer. The system must detect potholes, stray animals, and accidents from CCTV or uploaded video, calculate severity / risk, track before and after pothole state, attach camera geo-location, store incidents in Neon, send SMTP email alerts, and display a real-time authority dashboard. Use only free or open-source tools, avoid Google Maps / paid APIs / billing-required services, and keep the implementation simple, deterministic, and hackathon-ready.

14. Final Notes

- This document is intentionally strict so a fresh development chat can follow it without guessing the stack.
- If the implementation needs to be simplified further, keep the database, dashboard, and alert contract stable first.
- Do not add new libraries unless they clearly reduce complexity.

14. Added Enhancement: Night-Time Detection Capability

To ensure full coverage of system requirements and enhance real-world reliability, night-time detection capability is integrated into the AI pipeline without altering existing architecture.

Technical Enhancements:

- Low-light preprocessing using histogram equalization and noise reduction.
- Optional grayscale and contrast enhancement before YOLO inference.
- Model fine-tuning using low-light and night-time datasets (recommended: augmentations).
- Adaptive confidence thresholds for night conditions.

Where It Applies:

- Pothole Detection: Improved visibility of road surface defects under poor lighting.
- Animal Detection: Better tracking and recognition of animals during night hours.
- Accident Detection: Enhanced motion tracking and collision detection in low visibility.

Impact:

- Ensures 24/7 system reliability.
- Reduces false negatives in night scenarios.
- Aligns system with real-world deployment conditions.